



PROGRAMA ACADÉMICO DE INGENIERÍA EN SISTEMAS
ASIGNATURA: DISEÑO DE SISTEMAS EN INTERNET

Docente: Msc. Walter Quintero Acevedo

Semana 8 / Sesión 14 a16

Unidad Temática: Unidad: IV -
Tecnologías BACKEND && Despliegue y
seguimiento de cambios

Tema Central: Tecnologías BACKEND &&
Despliegue y seguimiento de cambios

Asignatura: Diseño de Sistemas en Internet

Profesor: MSc. Walter Quintero Nivel: Quinto Año

Material de Estudio: Unidad IV y V - Tecnologías Backend e Integración de Servicios Web

SECCIÓN 1: ARQUITECTURA BACKEND, FORMULARIOS Y SESIONES SEGURAS

1.1 Introducción

En la Ingeniería Web moderna, el Backend constituye el motor de procesamiento de cualquier sistema de información. Mientras el Frontend presenta la interfaz, el Backend es responsable de la lógica de negocio, la persistencia de datos, y crucialmente, de garantizar la seguridad y el estado de las interacciones del usuario. En esta sección, abandonamos los paradigmas monolíticos donde el servidor renderizaba HTML completo, y adoptamos la Arquitectura MVC (Modelo-Vista-Controlador) orientada a exponer datos, gestionando el estado mediante tokens seguros (JWT) en lugar de sesiones en memoria, lo cual es el estándar de escalabilidad en 2026.

1.2 Definiciones Fundamentales

- Backend: Capa de software que se ejecuta en el servidor. No produce HTML directamente (en la web moderna), sino que expone servicios de datos (APIs) que el frontend consume.
- Procesado de Formularios del Lado del Servidor: Recepción de payloads (cargas de datos) típicamente en formato JSON a través de peticiones HTTP POST/PUT. El servidor debe validar, sanitizar (prevenir inyecciones XSS/SQL) y procesar la información.



ÁREA DE CONOCIMIENTO: TECNOLOGÍA DE LA INFORMACIÓN Y LA COMUNICACIÓN

- Sesiones y Manejo de Estado: HTTP es un protocolo *stateless* (sin estado). Para saber quién es el usuario entre petición y petición, existen dos enfoques:
 - ✓ *Sesiones Cookies*: El servidor guarda el estado en memoria/BD y envía una cookie al cliente. (Ineficiente para microservicios/horizontalización).
 - ✓ *Tokens JWT (JSON Web Tokens)*: El servidor genera un token firmado criptográficamente tras el login. El cliente lo almacena y lo envía en cada petición. El servidor no guarda estado; valida la firma del token. (*Enfoque que aplicaremos*).
- Arquitectura MVC en el Backend:
 - ✓ Modelo (M): Define la estructura de los datos y la lógica de negocio directa con la Base de Datos (ej. Mongoose ORM, Prisma, Sequelize).
 - ✓ Vista (V): En una API REST, la "Vista" es la representación del recurso, típicamente la serialización JSON de los datos.
 - ✓ Controlador (C): Orquesta la lógica. Recibe la petición HTTP, invoca al Modelo, y devuelve la Vista (JSON) con el código de estado HTTP correspondiente.

1.3 Caso de Estudio Resuelto: Registro y Login Seguro con MVC y JWT

Contexto: Debemos implementar el módulo de autenticación para el sistema de Gestión de Tesis de la UNI. Herramientas a utilizar: Node.js, Express.js, bcryptjs (para hash de contraseñas), jsonwebtoken, Express-Validator.

Paso 1: Estructura de Carpetas (Patrón MVC) Creamos la siguiente arquitectura limpia:

/src

/controllers (Lógica de negocio)

/models (Esquemas de Base de Datos)

/routes (Definición de endpoints)

/middlewares (Validaciones y verificación de tokens)

Paso 2: El Modelo (Model) - **models/User.js** Definimos cómo se guarda un usuario. La contraseña jamás se guarda en texto plano.



javascript

```
const mongoose = require('mongoose');
```

```
const bcrypt = require('bcryptjs');
```

```
const userSchema = new mongoose.Schema({  
  nombre: { type: String, required: true },  
  email: { type: String, required: true, unique: true },  
  password: { type: String, required: true }  
});
```

```
// Hash de password antes de guardar
```

```
userSchema.pre('save', async function(next) {  
  if (!this.isModified('password')) return next();  
  const salt = await bcrypt.genSalt(10);  
  this.password = await bcrypt.hash(this.password, salt);  
  next();  
});
```

```
module.exports = mongoose.model('User', userSchema);
```

Paso 3: El Controlador (Controller) - **controllers/authController.js** Aquí procesamos el "formulario" (JSON entrante) y generamos la sesión (Token).

javascript

```
const User = require('../models/User');
```

```
const jwt = require('jsonwebtoken');
```

```
// Procesado de Formulario de Registro
```

```
exports.register = async (req, res) => {  
  // Se asume que express-validator sanitizó req.body previamente  
  const { nombre, email, password } = req.body;
```



```
try {
  let user = await User.findOne({ email });
  if (user) return res.status(400).json({ msg: 'El usuario ya existe' });

  user = new User({ nombre, email, password });
  await user.save(); // El pre-hook del modelo hashlea la password

  return res.status(201).json({ msg: 'Usuario creado exitosamente' });
} catch (error) {
  res.status(500).json({ error: 'Error del servidor' });
}
};

// Manejo de Sesiones (Login) -> Generación de JWT
exports.login = async (req, res) => {
  const { email, password } = req.body;
  try {
    const user = await User.findOne({ email });
    if (!user) return res.status(400).json({ msg: 'Credenciales inválidas' });

    // Verificar password hasheada
    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) return res.status(400).json({ msg: 'Credenciales inválidas' });

    // Crear Sesión (JWT)
    const payload = { userId: user.id };
    const token = jwt.sign(payload, process.env.JWT_SECRET, { expiresIn: '2h' });

    res.json({ token }); // Enviamos el estado al cliente
  }
};
```



```
    } catch (error) {  
      res.status(500).json({ error: 'Error del servidor' });  
    }  
  };  
};
```

Paso 4: Middleware de Sesión Segura - **middlewares/authMiddleware.js** Para proteger rutas, verificamos que el token JWT sea válido.

javascript

```
const jwt = require('jsonwebtoken');  
  
exports.verifyToken = (req, res, next) => {  
  const header = req.header('Authorization');  
  if (!header) return res.status(401).json({ msg: 'Acceso denegado. No hay token' });  
  
  try {  
    const token = header.replace('Bearer ', '');  
    const decoded = jwt.verify(token, process.env.JWT_SECRET);  
    req.user = decoded; // Inyectamos el usuario en la petición  
    next(); // Pasa al controlador  
  } catch (error) {  
    res.status(401).json({ msg: 'Token inválido o expirado' });  
  }  
};
```

1.4 Tareas y Ejercicios Independientes

1. Implementación: Monte el código anterior en su entorno local. Cree las rutas en Express (**/api/auth/register** y **/api/auth/login**) conectándolas a sus respectivos controladores.
2. Protección de Ruta: Cree un endpoint **/api/auth/profile** que retorne los datos del usuario logueado. Proteja esta ruta usando el **verifyToken** creado.



3. Validación de Formularios: Investigue e implemente la librería **express-validator** en la ruta de registro para asegurar que el **email** tenga formato válido y la **password** tenga mínimo 8 caracteres.

SECCIÓN 2: SERVICIOS WEB Y CREACIÓN DE APIs RESTful

2.1 Introducción

La web ha evolucionado de ser un conjunto de páginas interconectadas a una plataforma de servicios interoperables. Un Sistema de Información Web moderno no vive aislado; expone su lógica de negocio para que sea consumida por aplicaciones móviles, otros sistemas o SPAs (Single Page Applications). En esta sección, construiremos el contrato de datos de nuestro sistema mediante una API RESTful, reemplazando los obsoletos Web Services SOAP por la ligereza y velocidad de JSON sobre HTTP.

2.2 Definiciones Fundamentales

- Servicios Web SOAP (Simple Object Access Protocol): Protocolo legacy fuertemente tipado que usa XML con un "sobre" (envelope) estricto. Requiere WSDL para definir el contrato. Su overhead (sobrecarga) lo hace ineficiente para la web moderna de alta concurrencia.
- Servicios Web REST (Representational State Transfer): Estilo arquitectónico que utiliza los verbos nativos de HTTP (GET, POST, PUT, DELETE) sobre recursos (URLs) para operar, respondiendo generalmente en JSON. Es *stateless* (sin estado) y altamente escalable.
- Tecnologías Subyacentes:
 - ✓ HTTP: Protocolo de transporte. Define la intención de la operación (Verbos).
 - ✓ JSON (JavaScript Object Notation): Formato de intercambio de datos ligero, legible por humanos y de parseo nativo en JavaScript, reemplazando a XML.



ÁREA DE CONOCIMIENTO: TECNOLOGÍA DE LA INFORMACIÓN Y LA COMUNICACIÓN

- ✓ API RESTful: Implementación práctica de REST. Un endpoint como **/api/usuarios/123** representa la entidad, y el Verbo HTTP define la acción.

2.3 Caso de Estudio Resuelto: CRUD de Productos y Pruebas con Postman

Contexto: El sistema de Gestión de Tesis necesita un módulo para administrar "Líneas de Investigación" (Productos/Recursos). Herramientas a utilizar: Node.js, Express, Postman (Para evaluación formativa).

Paso 1: El Modelo - **models/LineaInvestigacion.js**

javascript

```
const mongoose = require('mongoose');
const lineaSchema = new mongoose.Schema({
  nombre: { type: String, required: true },
  codigo: { type: String, required: true, unique: true },
  activa: { type: Boolean, default: true }
});
module.exports = mongoose.model('LineaInvestigacion', lineaSchema);
```

Paso 2: El Controlador - **controllers/lineaController.js** Implementamos el CRUD estándar de un servicio web RESTful.

javascript

```
const Linea = require('../models/LineaInvestigacion');
```

```
// GET - Leer todos los recursos
```

```
exports.obtenerLineas = async (req, res) => {
  const lineas = await Linea.find();
  res.json(lineas);
};
```

```
// POST - Crear un recurso
```

```
exports.crearLinea = async (req, res) => {
  const nuevaLinea = new Linea(req.body);
```



```
    await nuevaLinea.save();  
    res.status(201).json(nuevaLinea);  
};
```

// PUT - Actualizar un recurso existente

```
exports.actualizarLinea = async (req, res) => {  
    const linea = await Linea.findByIdAndUpdate(req.params.id, req.body, { new: true });  
    if (!linea) return res.status(404).json({ msg: 'No encontrada' });  
    res.json(linea);  
};
```

// DELETE - Eliminar un recurso

```
exports.eliminarLinea = async (req, res) => {  
    await Linea.findByIdAndDelete(req.params.id);  
    res.json({ msg: 'Línea eliminada' });  
};
```

Paso 3: Las Rutas (Definición de la API) - **routes/lineaRoutes.js** Mapeamos los verbos HTTP a nuestros controladores.

javascript

```
const express = require('express');  
const router = express.Router();  
const ctrl = require('../controllers/lineaController');  
const auth = require('../middlewares/authMiddleware');
```

```
// Protegemos las rutas de modificación con el JWT  
router.get('/', auth.verifyToken, ctrl.obtenerLineas);  
router.post('/', auth.verifyToken, ctrl.crearLinea);  
router.put('/:id', auth.verifyToken, ctrl.actualizarLinea);  
router.delete('/:id', auth.verifyToken, ctrl.eliminarLinea);
```



module.exports = router;

Paso 4: Evaluación Formativa con Postman

1. En Postman, cree una colección "API Tesis UNI".
2. Haga un POST a **/api/auth/login** y guarde el token devuelto.
3. En las variables de la colección, guarde el token: **Bearer {{token}}**.
4. Configure el Header **Authorization** con este token para todas las peticiones subsiguientes.
5. Prueba CRUD:
 - ✓ **POST /api/lineas** con body **{"nombre": "Inteligencia Artificial", "codigo": "IA-01"}** -> Esperado: Status 201 Created.
 - ✓ **GET /api/lineas** -> Esperado: Array JSON con la línea creada.
 - ✓ **PUT /api/lineas/{id}** -> Esperado: Recurso actualizado.
 - ✓ **DELETE /api/lineas/{id}** -> Esperado: Mensaje de confirmación.

2.4 Tareas y Ejercicios Independientes

1. Extensión del CRUD: Implemente el CRUD completo para la entidad **Estudiante** (con campos: carnet, nombre, carrera). Exponga los endpoints correspondientes.
2. Relaciones: Modifique el modelo **Estudiante** para que tenga un campo **lineaInvestigacion** que sea una referencia (ObjectId) al modelo **LineaInvestigacion**. Implemente un endpoint GET que devuelva un estudiante con el nombre de su línea de investigación poblada (use **.populate()** de Mongoose).
3. Postman: Exporte su colección de Postman con las pruebas exitosas y súbala al repositorio de su proyecto final.



SECCIÓN 3: CONSUMO DE APIs E INTEGRACIÓN FULL-STACK

3.1 Introducción

El paradigma de la Web Moderna (Arquitectura Desacoplada) alcanza su máxima expresión cuando el Frontend y el Backend son entidades independientes que comunican a través de un contrato: la API REST. En esta sección, transformaremos nuestro Frontend estático o dependiente de recargas de página (Server-Side Rendering clásico) en un cliente dinámico que consume datos asíncronamente, proporcionando una experiencia de usuario fluida comparable a una aplicación de escritorio (SPA).

3.2 Definiciones Fundamentales

- Consumo de Servicios Web: El acto de realizar peticiones HTTP desde el código que se ejecuta en el navegador del cliente hacia la API Backend.
- Fetch API: Interfaz nativa de los navegadores modernos (estándar ECMAScript) para realizar peticiones HTTP asíncronas basadas en Promesas, reemplazando al obsoleto **XMLHttpRequest**.
- Axios: Librería de terceros (muy usada con frameworks como React/Vue) que funciona como un wrapper sobre Fetch, ofreciendo interceptores de peticiones, transformación automática de JSON a objetos JS, y mejor manejo de errores.
- Integración Frontend-Backend: El flujo donde la UI (Vista) dispara un evento -> Llama a la API (Fetch/Axios) -> Recibe JSON -> Actualiza el DOM (Document Object Model) sin recargar la página.
- CORS (Cross-Origin Resource Sharing): Mecanismo de seguridad del navegador que bloquea peticiones a dominios diferentes al de origen. El Backend *debe* configurar cabeceras CORS para permitir que el Frontend (que corre en localhost:3000, por ejemplo) consuma la API (que corre en localhost:5000).

3.3 Caso de Estudio Resuelto: Integración del Login y CRUD en el Frontend

Contexto: Tenemos una página HTML simple con un formulario de Login y una sección para mostrar las Líneas de Investigación. Consumiremos la API creada en la Sección 2. Herramientas a utilizar: HTML5, JavaScript Vanilla (ES6+), Fetch API, servidor Backend Express (con cors habilitado).



Paso 1: Habilitar CORS en el Backend En **server.js** (Backend), antes de las rutas:

javascript

```
const cors = require('cors');  
app.use(cors({ origin: 'http://127.0.0.1:5500' })); // Origen del Frontend
```

Paso 2: Login y Almacenamiento de Sesión (Token) en el Frontend

javascript

// login.js

```
document.getElementById('formLogin').addEventListener('submit', async (e) => {  
    e.preventDefault(); // Prevenir recarga de la página
```

```
    const email = document.getElementById('email').value;  
    const password = document.getElementById('password').value;
```

```
    try {  
        const response = await fetch('http://localhost:5000/api/auth/login', {  
            method: 'POST',  
            headers: { 'Content-Type': 'application/json' },  
            body: JSON.stringify({ email, password })  
        });
```

```
        if (!response.ok) throw new Error('Credenciales incorrectas');
```

```
        const data = await response.json();
```

```
        // Guardar el token de sesión de forma segura  
        localStorage.setItem('jwt_token', data.token);
```

```
        // Redirigir o cargar vista principal sin recargar  
        cargarLineasInvestigacion();
```



```
    } catch (error) {  
        alert(error.message);  
    }  
});
```

Paso 3: Consumo del CRUD (GET) e Inyección en el DOM

javascript

// lineas.js

```
async function cargarLineasInvestigacion() {  
    const token = localStorage.getItem('jwt_token');  
  
    try {  
        const response = await fetch('http://localhost:5000/api/lineas', {  
            method: 'GET',  
            headers: {  
                'Authorization': `Bearer ${token}` // Enviamos la sesión  
            }  
        });  
    };  
  
    if (response.status === 401) {  
        // Token expirado o inválido  
        window.location.href = '/login.html';  
        return;  
    }  
  
    const lineas = await response.json();  
  
    // Renderizar en el DOM sin recarga de página  
    const contenedor = document.getElementById('lista-lineas');  
    contenedor.innerHTML = ""; // Limpiar contenedor
```



```
lineas.forEach(linea => {  
  contenedor.innerHTML += `  
    <div class="card">  
      <h3>${linea.nombre}</h3>  
      <p>Código: ${linea.codigo}</p>  
      <button onclick="eliminarLinea('${linea._id}')">Eliminar</button>  
    </div>  
  `;  
});  
} catch (error) {  
  console.error('Error al cargar líneas:', error);  
}  
}
```

Paso 4: Consumo del CRUD (DELETE) de forma asíncrona

javascript

```
async function eliminarLinea(id) {  
  const token = localStorage.getItem('jwt_token');  
  const confirmar = confirm('¿Está seguro de eliminar esta línea?');  
  if (!confirmar) return;  
  
  try {  
    const response = await fetch(`http://localhost:5000/api/lineas/${id}`, {  
      method: 'DELETE',  
      headers: { 'Authorization': `Bearer ${token}` }  
    });  
  
    if (response.ok) {  
      // Éxito: Actualizamos la vista quitando el elemento del DOM
```



```
// Sin hacer un reload de la página completa
alert('Línea eliminada');
cargarLineasInvestigacion(); // Refrescamos la lista vía API
}
} catch (error) {
  alert('Error al eliminar');
}
}
```

3.4 Tareas y Ejercicios Independientes

1. Integración Total: Convierta las vistas estáticas de su Proyecto Final (hechas en Fase 1) en vistas dinámicas. Conecte su Frontend a su Backend usando **Fetch API** o **Axios** (si usa React/Vue).
2. Manejo de Estados de Carga: Cuando se llama a una API, hay un retraso (latencia). Implemente un "Spinner" (Indicador de carga) que se muestre en el Frontend justo después del **fetch()** y se oculte cuando los datos lleguen y se rendericen en el DOM.
3. Formularios Asíncronos: Implemente la creación de una "Línea de Investigación" desde el Frontend. Capture el formulario con JavaScript, envíelo por POST a la API, y en caso de éxito (201), agregue el nuevo elemento dinámicamente a la lista en pantalla sin recargar la página.
4. Cierre de Sesión: Implemente un botón de "Logout" que elimine el token del **localStorage** y redirija al usuario a la pantalla de Login.